

● БОЛЬШИЕ ВЫЗОВЫ 2025-2026

Agent AI

Разработка нейросетевой программной среды с динамической
само модификацией

НАПРАВЛЕНИЕ



Большие данные, ИИ
и информационная безопасность

РАЗРАБОТЧИК



Ролдугин Илья Владимирович
ГБПОУ ВО "ГПК"

Актуальность, Цели и Задачи



Актуальность

Индустрия переходит от простых чат-ботов к **автономным агентным системам**, способным самостоятельно решать сложные задачи.



Проблема

Существующие ИИ-клиенты имеют **статичную архитектуру**: их инструменты жестко заданы разработчиком и не меняются под задачу.



ЦЕЛЕВАЯ АУДИТОРИЯ

IT-специалисты и опытные пользователи



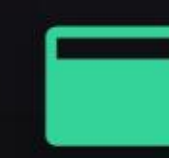
Цель проекта

Создание высокопроизводительного ИИ-агента с архитектурой динамической самомодификации.

КЛЮЧЕВЫЕ ЗАДАЧИ



Механизм Runtime-исполнения и модификации кода.



Адаптивный UI (пресеты и HTML-вставки).



Взаимодействие ИИ с GUI операционной системы.



Автономное повышение качества кодинга модели.

Ядро Системы и Базовые Инструменты



MAIN.PY

```
from agent import Chat

ai = Chat()
ai.set_preset("developer")
ai.set_mode("file_structure", "sandbox")

ai.add_message(f"Задача пользователя: {input()}")
files = ai.ai_get(
    "Выведи список файлов для изменения",
    target_type=list,
    clear_history=False
)
ai.send("Выполни задачу, меняя файлы внутри песочницы")

# ...
```



Центральный узел

Класс **Chat** — это ядро агента. Он управляет API нейросети, хранит историю контекста, предоставляет доступ к инструментам и поддерживает локальное окружение (`self.local_env`).

БАЗОВЫЙ ИНСТРУМЕНТАРИЙ ОС



Инструмент Python

Исполнение сгенерированного кода напрямую в локальном окружении процесса.



Инструмент Shell

Интерактивная сессия терминала с сохранением контекста между вызовами.

Динамическая Самомодификация

АРХИТЕКТУРА ИЗОЛЯЦИИ (SANDBOX)

СЛОВАРЬ LOCAL_ENV



Экземпляр Chat
self



Переменная вывода

result = ""

AGENT.PY (УПРОЩЕННАЯ РЕАЛИЗАЦИЯ ФРАГМЕНТА)

```
def python_tool(self, code):
    self.local_env["self"] = self
    self.local_env["result"] = ''
    exec(code, self.local_env, self.local_env)
    return str(self.local_env["result"])
```

ВЫЗОВ ИНСТРУМЕНТА PYTHON

```
import types

# 1. Определение логики инструмента
def custom_math_tool(self, expression):
    # Безопасный вызов eval без builtins
    res = eval(expression, {'__builtins__': {}})
    return f"Результат вычисления: {res}"

# 2. Привязка метода к экземпляру Chat
self.custom_math_tool = types.MethodType(
    custom_math_tool, self
)

# 3. Регистрация JSON-схемы для Gemini
self.tools.append({
    # Описание в формате OpenAI API
})
```


Вспомогательные Инструменты Ядра

python_str

99

Специализированный инструмент форматирования. Принимает текст и возвращает его как Python-строку, **экранируя спецсимволы**. Критически важен для безопасной записи кода в файлы.

google_search



Интеграция с **Google Custom Search API**. Позволяет агенту самостоятельно искать актуальную информацию в интернете, компенсируя устаревание весов модели.

sandbox



Управление локальной песочницей. Копирует проект во временную папку и **запускает изолированный процесс сервера**. Позволяет агенту безопасно тестировать свои изменения.

http



Загружает веб-страницы по URL, **очищает HTML от тегов и скриптов** с помощью BeautifulSoup, возвращая агенту чистый, легко анализируемый текстовый контент.

user_profile



Система **долгосрочной памяти**. Сохраняет в JSON личные предпочтения пользователя, профессиональный контекст и общие инструкции, обеспечивая непрерывность между сессиями.

Система Пресетов, Промптов и ACL

ДИНАМИЧЕСКОЕ УПРАВЛЕНИЕ

Система пресетов позволяет **мгновенно перенастраивать агента** под задачу. При смене пресета меняются:

-  **Системные промпты**
Глобальные текстовые инструкции, задающие общую "личность", правила поведения и контекст (например, "Ты - Архитектор, планируй...").
-  **Режимы**
Добавляют свой промпт при включении, так же позволяют настроить ACL и запустить скрипт сбора данных.
-  **Команды**
Отправляют свой промпт при нажатии, так же позволяют запустить скрипт.
-  **Доступ к инструментам**
Например: можно заблокировать python.
-  **Списки контроля доступа (ACL)**
Жесткий перехват файловых операций на уровне Python (sys.addaudithook) для защиты проекта от разрушительных действий ИИ.

ПРИМЕРЫ ВСТРОЕННЫХ ПРЕСЕТОВ



«Безопасный»

Максимальная защита системы. Агенту **запрещено изменять любые файлы проекта** (ACL: только чтение). Запись разрешена исключительно в изолированные директории temp/ и sandbox/.



«Разработчик»

Полный доступ к коду. Через *режимы* можно динамически включать исходники файлов в контекст по разделам (frontend, backend, plugins), чтобы агент **знал содержимое файлов, а не угадывал**.



«Ревьюер»

Жесткое ограничение на редактирование (ACL: Read-Only). Иницируется специальный промпт для **глубокого статического анализа качества кода**, поиска уязвимостей и архитектурных багов без риска поломки проекта.

Плагин web interface



ПОЛЬЗОВАТЕЛЬСКИЙ ОПЫТ

Полноценный Web-GUI



Современный интерфейс

Сервер (localhost) предоставляет адаптивный интерфейс на базе **Vue.js** и **Tailwind CSS**. Встроенные парсеры (Marked.js, Highlight.js, KaTeX) обеспечивают идеальное отображение кода, математики и markdown.



Сериализация сессий

Чаты сохраняются не просто как текст, а **сериализуются в бинарный формат (dill)**. Это позволяет сохранять сгенерированные в рантайме методы, переменные окружения и текущее состояние агента.



Визуальное управление

Через боковую панель можно **редактировать пресеты и промпты**, а управлять **режимами** и запускать **команды** прямо из панели ввода чата.

Напишите что-нибудь...



ОЧИСТИТЬ ПРОФИЛЬ



КОММИТ



↑
РЕЖИМЫ

↑
КОМАНДЫ

Интерактивная Визуализация



ОТКАЗ ОТ СУХОГО ТЕКСТА

HTML-вставки на лету



Tailwind CSS & JavaScript

Агент генерирует **полноценные UI-компоненты**. Теги не экранируются, позволяя создавать дашборды, графики и сложную верстку прямо в потоке чата.



Интерактивное управление

Вместо текстовых подтверждений используются **интерактивные кнопки и формы** обратной связи, вызывающие JS-функции (например, отправку следующей команды агенту).

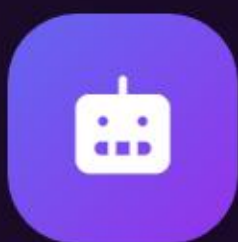


Защищенный рендеринг LaTeX

Кастомный Markdown-парсер изолирует математические формулы (**KaTeX**), гарантируя, что сложные уравнения не сломают разметку страницы.



UI-ВИДЖЕТЫ, СГЕНЕРИРОВАННЫЕ ИИ



Я проанализировал логи сервера. Процесс парсинга завершен. Вот текущий статус и доступные действия:



УСПЕШНО

1,402



ОШИБОК

3

Генерация финального отчета

75%

Ожидание подтверждения

Проверьте ошибки вручную или позвольте мне попробовать их исправить (Auto-Fix).

 Auto-Fix

 Показать логи

Качество Кода: Проблема Масштаба

ИИ отлично решает атомарные задачи. Но при попытке создать **объемный продукт за один подход** (Single-shot), качество катастрофически падает из-за наложения следующих факторов:



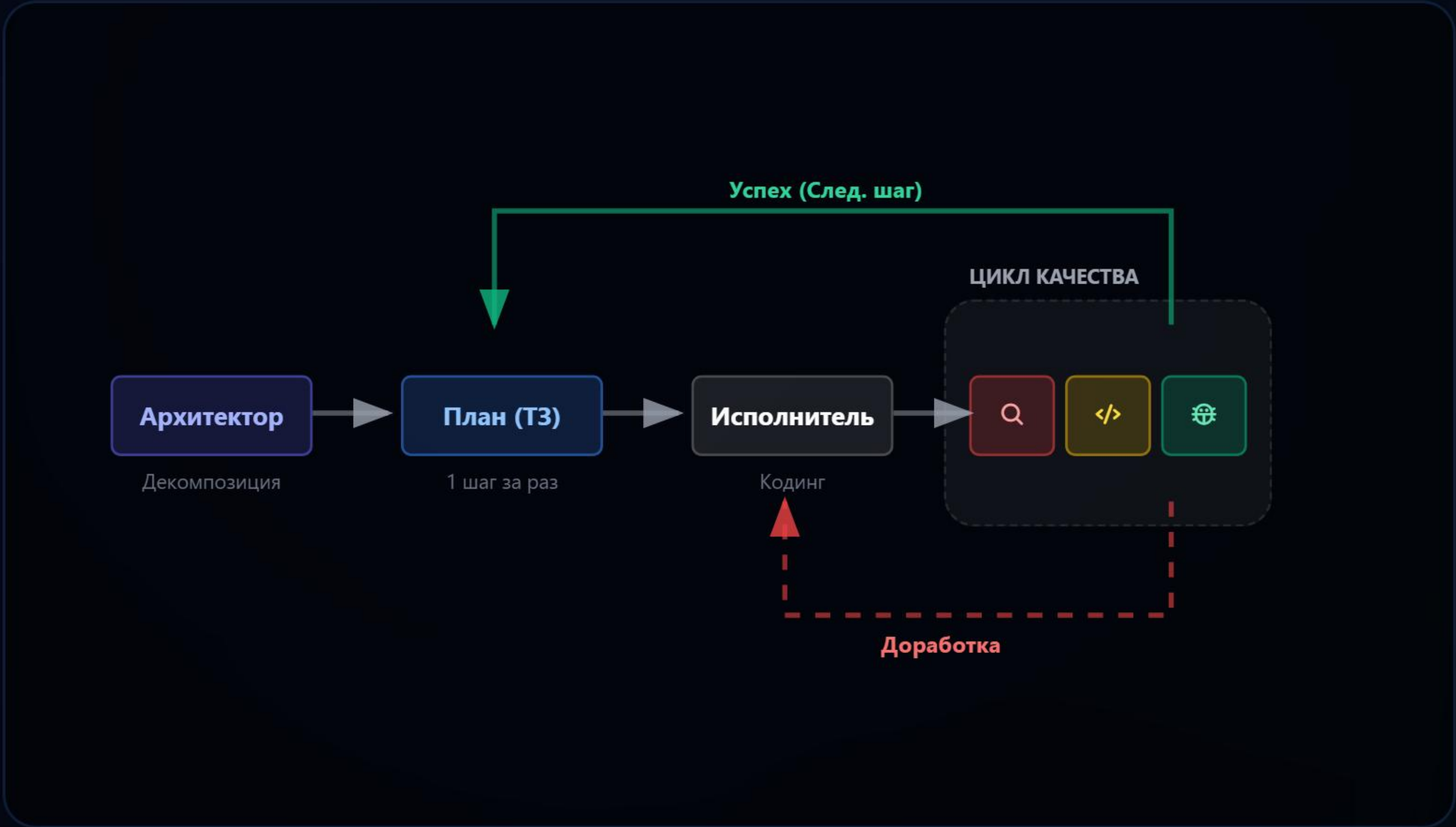
Решение проблемы: Эволюция подходов



УСТАРЕВШИЙ ПОДХОД

Режим «Архитектор»

Многоагентный подход: дробление задачи на разные чаты.



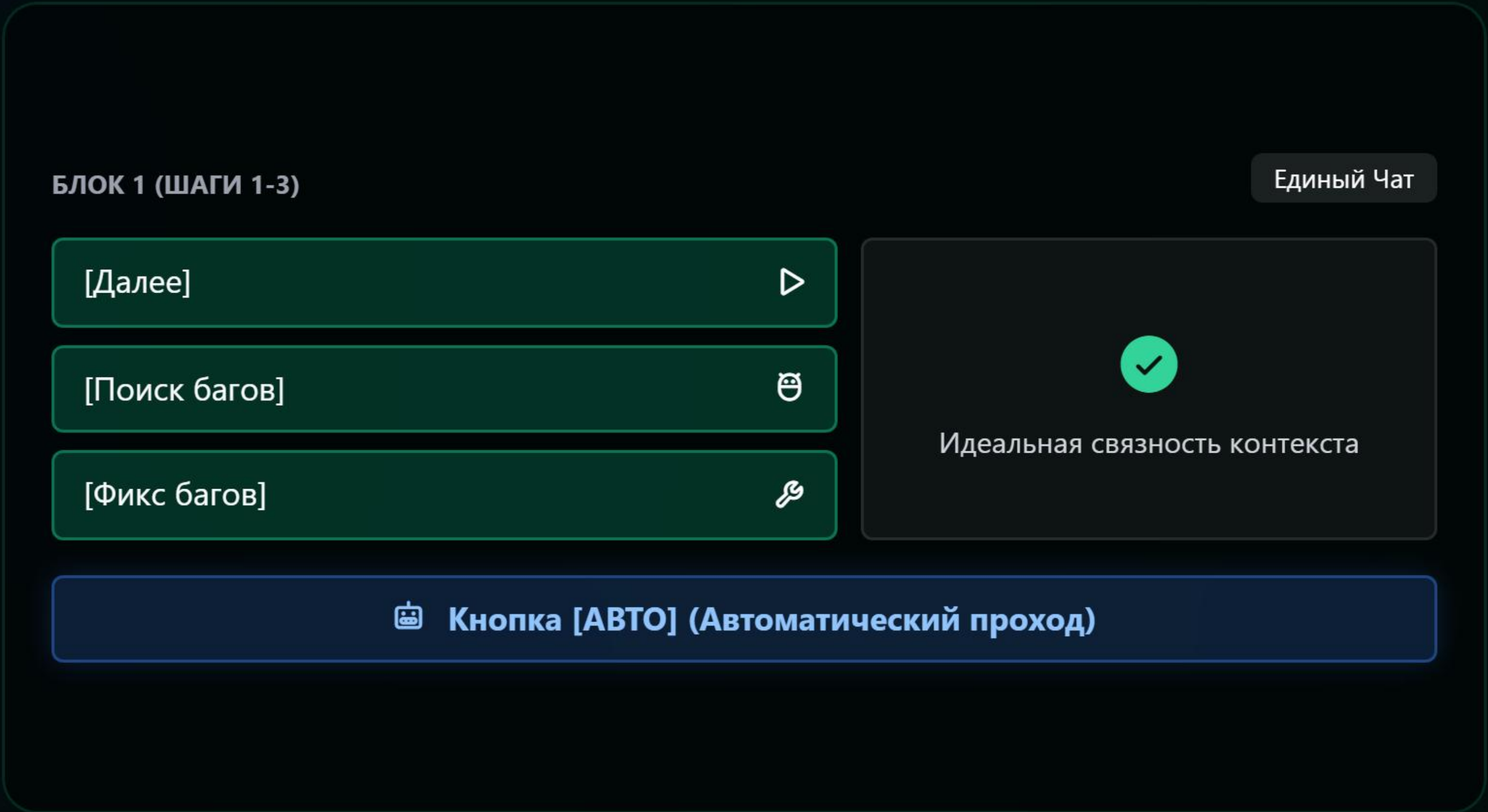
VS



АКТУАЛЬНОЕ РЕШЕНИЕ

Режим «Проектировщик»

Single-Chat & Auto-Prompts: все действия в пределах одного контекста, управляемые жестким скриптом кнопок.



ФАТАЛЬНЫЕ НЕДОСТАТКИ

- ✗ **Галлюцинации проверяющих:** выдумывают ошибки, которых нет.
- ✗ **Рассинхрон контекста:** чаты не видят полную картину друг друга.
- ✗ **Затраты токенов:** дублирование входного контекста.

КЛЮЧЕВЫЕ ПРЕИМУЩЕСТВА

- ✓ **Отсутствие рассинхрона:** агент видит всё, что делал ранее.
- ✓ **Высокое качество:** значительное улучшение финального результата работы модели.

Взаимодействие с GUI: Браузер



Проблема "Слепоты"

Языковые модели отлично пишут скрипты, но абсолютно беспомощны при попытке выполнить простейшие задачи в

Графическом Интерфейсе Пользователя (GUI), особенно на веб-сайтах.

ПУТИ РЕШЕНИЯ

1

Передача сжатого **DOM-дерева** (Подходит для веба)

2

Мультимодальные модели (VLM) + Скриншоты



РЕАЛИЗАЦИЯ ПОДХОДА №1

Плагин Browser Use

Специализированное **браузерное расширение** извлекает из страницы текст и индексирует интерактивные элементы (кнопки, ссылки). Агент управляет страницей через инъекцию JavaScript по уникальным ID элементов.

`{ }` СТРУКТУРА ВЫВОДА BROWSER_GET_DOM

```
"title": "Page", "url": "https://..."  
"fullText": "Весь текст (вкл. IFRAME) для поиска данных"  
"interactive": [  
  { tag: "button", text: "Login", id: "btn-log", class: "btn" },  
  { tag: "input", text: "", id: "user", class: "input" }  
]
```

✓ Только видимые элементы (скрытые фильтруются)

🎯 ID — самый надежный способ для селектора

Взаимодействие с GUI: Computer Use



РЕЖИМ НАТИВНЫЙ

Windows Base

Агент управляет **реальной хост-системой** пользователя через эмуляцию ввода (`pyautogui`, `mss`).

+ Не требует дополнительной настройки (работает "из коробки").
Доступ ко всем локальным приложениям.

- Во время работы **нельзя пользоваться ПК** (движение мыши собьет скрипт).



РЕЖИМ ИЗОЛИРОВАННЫЙ

Виртуальный (WSL)

Создается виртуальный дисплей (`Xvfb`) внутри подсистемы Linux. Управление через `xdotool`, трансляция VNC.

+ Агент работает **в фоне**, не перехватывая курсор.
Полностью безопасная среда для тестирования кода ИИ.

- Требуется предварительная настройка WSL и пакетов.
Недоступны специфичные Windows-приложения.

Тестирование Системы



Ядро и Самомодификация

- ✓ Создание инструмента "на лету"
- ✓ Сериализация состояния
- ✓ Защита ACL (перехват sys.addaudithook)



Качество Кода

Задача: Добавить в web ui возможности персонализации

Обычный режим

Провал (регрессия кода)

Режим "Проектировщик"

Успех (есть мелкие баги, но нет регрессии)



Базовые инструменты

- ✓ Изолированная песочница (sandbox)
- ✓ Интерактивный терминал (shell)
- ✓ Поиск и парсинг Web (google, http)



Взаимодействие с GUI

- ✓ Простой Web: Browser Use ("РешуЕГЭ")
- ✓ Сложный Web: Computer Use WSL (React)
- ⚠ Native ОС: Успешно, но требует описания интерфейса (Paint)

Сравнительный Анализ

 СУЩЕСТВУЮЩИЕ АВТОНОМНЫЕ AI-АГЕНТЫ

КРИТЕРИЙ	AGENT AI	OPENHANDS	OPEN INTERPRETER	SWE-AGENT / DEVIN
Архитектура	Самомодаификация	Статичная (Скрипты)	Статичная (Ядро неизменно)	Жестко заданная
Окружение	Прямой доступ к ОС + ACL изоляция	Изолированный Docker	Прямой доступ к ОС (CLI)	Облачная / Изолированная
Пользовательский интерфейс	Web UI с HTML-вставками	Статичный React-UI	Терминал	Web-UI / Терминал
Взаимодействие с GUI	Нативное (PyAutoGUI) + Изолированное (WSL) + Анализ DOM (Browser Use)	VNC внутри Docker	Нативное (PyAutoGUI)	Не поддерживается

 Главные преимущества: Самомодаификация в Runtime, HTML-вставки и комплексность.

Будущее Автономной Разработки



Результаты

- ✓ Доказана работоспособность концепции самомодификации.
- ✓ Создан инновационный динамический **Web UI**.
- ✓ Решена проблема интеграции агента с **GUI ОС** и Веба.



Перспективы

- Оптимизация промптов для снижения расхода токенов.
- Интеграция системы **RAG** для работы с большими базами данных.
- Инструменты для изменения **UI** агентом "на лету".

ИСХОДНЫЙ КОД НА GITHUB



ilia202015/AgentAI